

Programovací jazyky a překladače

Přednáška 2, poznámky

Jan Voráček

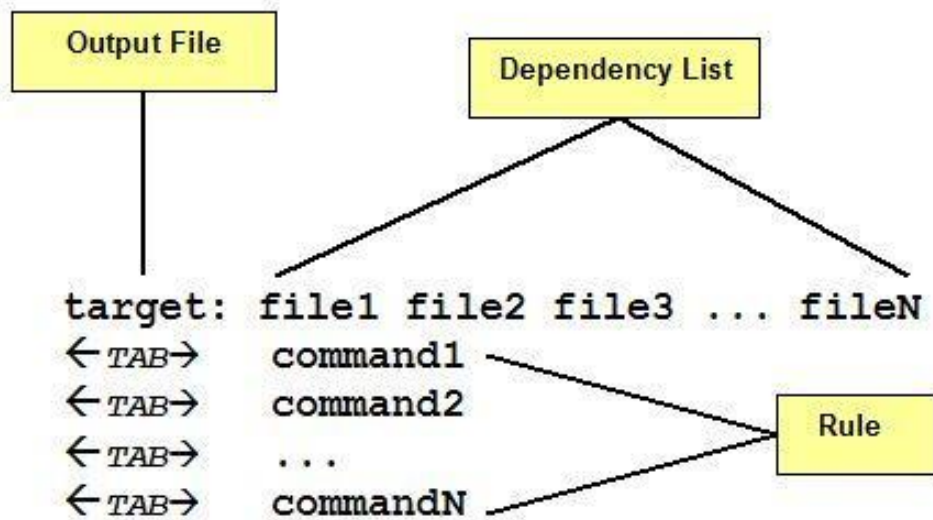
VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

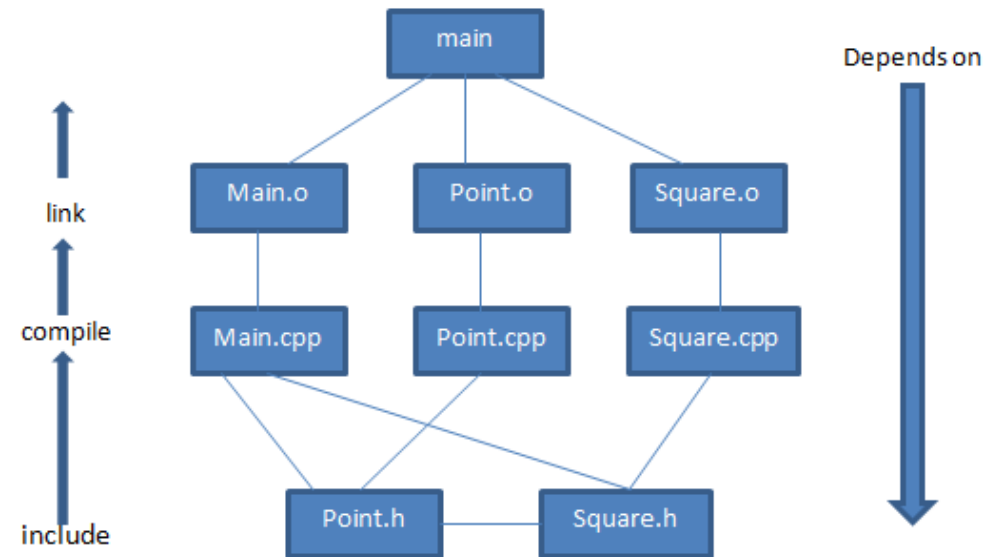
- Administrativa
 - Připomínky k výuce ani k použitému prostředí jsem zatím nezaznamenal
 - Bonusy z T1 zapsány do IS
 - Probíhá rezervace témat individuálních prezentací v PS
 - Nezapomeňte včas (= úterý, 12:00) odevzdávat své soubory
- Prezentace
 - Rozdíl mezi překladačem a interpretem
 - T diagramy
 - Regulární jazyky a výrazy
- Utilita make
- Generátor lexikálních analyzátorů LEX
 - Napřed trocha související teorie (make, RE, konečné automaty), zbytek později ...
 - Základy vývoje analyzátorů v lex_u
 - Praktické možnosti jejich generování

Make, základní principy

- Sestavení SW projektu na základě jeho blokové struktury (grafu závislosti, dependency graph) a časových relací mezi jednotlivými uzly.
- Základem činnosti je skriptový soubor (*Makefile*), jehož obsah je vykonám utilitou *make*.



Zdroj: <https://bit.ly/3yqnk0r>



Zdroj: <https://bit.ly/3yte7Vm>

Makefile, struktura souboru

Obecně:

cíl: všechny soubory, na kterých cíl závisí
<TAB> předpis k realizaci příkazu/sestavení souboru cíl

Konkrétněji:

celkový_cíl: dílčí_cíl_1 zdrojový_kód_pro_celkový_cíl_1.c
<TAB> předpis k realizaci příkazu celkový_cíl
dílčí_cíl_1: zdrojový_kód_pro_dílčí_cíl_1.c
<TAB> předpis k realizaci příkazu dílčí_cíl_1

Cíl (target) se provede, pokud neexistuje výstup, specifikovaný v příkazech za dvojtečkou nebo pokud je kterýkoli související zdrojový soubor mladší než existující výstup.

Prerequisite (dependency) za dvojtečkou cíle může být i jiný cíl, který se pak vyhledává shora dolů a vykonává zdola nahoru.

Jednoduché příklady

ahoj:

@echo "Ahoj"

edit : main.o

cc -o edit main.o

main.o : main.c defs.h

cc -c main.c

clean :

rm edit main.o

Sestavování SW projektů, utilita *make* pohledem ChatGPT

- Otázka: Example of makefile for Hello world program?
- Odpověď: Here's an example makefile for compiling a simple "Hello World" program in C:



```
CC=gcc
CFLAGS=-Wall
all: hello
hello: hello.c
    $(CC) $(CFLAGS) -o hello hello.c
clean:
    rm -f hello
```

Explanation:

- CC=gcc: specifies the compiler to be used (gcc in this case).
- CFLAGS=-Wall: sets the compiler flags to include warnings for all common programming errors.
- all: hello: declares that the target "all" depends on the target "hello", meaning that running "make all" will compile the "hello" target.
- hello: hello.c: declares that the target "hello" depends on the source file "hello.c".
- \$(CC) \$(CFLAGS) -o hello hello.c: compiles the "hello.c" source file into an executable called "hello", using the specified compiler and flags.
- clean: declares a target for cleaning up the directory by removing the executable file.
- rm -f hello: removes the executable file.
- To use this makefile, save it as a file named "Makefile" in the same directory as your "hello.c" source file. Then, simply run the command "make" in the terminal to compile the program. You can also run "make clean" to remove the executable file.

Make, proměnné a spuštění

- Makefile může obsahovat proměnné uživatelem definované ($\$<\text{jméno}>$) nebo automatické ($\$@$ = aktuální cíl, $\$<$ = první prerekvizita apod.)
- Spuštění :
 - make (hledá soubory makefile nebo Makefile v aktuálním adresáři)
 - make -f <vlastní jméno sestavovacího souboru>
 - Po spuštění je vykonán první cíl v pořadí, (může být navázán na další cíle) respektive příkazem make <jméno cíle> může být spuštěn pouze určený cíl
- Více informací:
 - <https://www.gnu.org/software/make/manual/make.html>
 - <https://makefiletutorial.com/>
 - <https://opensource.com/article/18/8/what-how-makefile>
 - chatGPT

Konečný automat (KA), definice dle doc. Beneše



Konečný automat

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q : (konečná) množina stavů
- Σ : vstupní abeceda
- δ : přechodová funkce

$$\delta_{NKA} : Q \times \Sigma \rightarrow 2^Q$$

$$\delta_{DKA} : Q \times \Sigma \rightarrow Q$$

- q_0 : počáteční stav
- F : množina koncových stavů

Konečný automat, vzorový příklad, více později ...

Mějme automat $M = (Q, T, d, q_0, F)$, kde:

$Q = \{q_0, q_1\}$

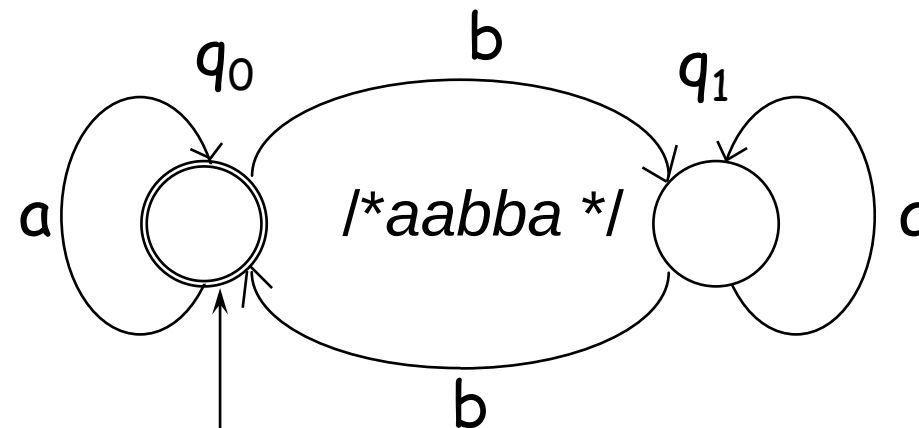
$T = \{a, b\}$

$F = q_0$

d je dáno tabulkou

q	s	$\delta(q,s)$			
q_0	a	q_0	δ	a	b
q_0	b	q_1	q_0	q_0	q_1
q_1	a	q_1	q_1	q_1	q_0
q_1	b	q_0			

Automat lze popsat také stavovým (přechodovým) diagramem:



Lexikální analýza

Miroslav Beneš
Dušan Kolář



Regulární výraz (Regular Expression, RE)

- Jeden ASCII znak
- Prázdný řetězec (ε)
- Spojení (zřetězení, konkatenace) dvou RE
 - Zápis: RE1 RE2 (RE1 následováno RE2)
- Sjednocení dvou RE
 - Zápis: RE1 | RE2
- Uzávěr RE
 - Zápis: RE*
 - Zřetězení 0 nebo více takto označených řetězců
- Nenulový výčet
 - Zápis: RE+
 - Zřetězení 1 nebo více takto označených řetězců

Další entity v jazyce RE

- **Skupiny** vyznačené kulatými závorkami (.....)
 - **Rozsahy** vyznačené hranatými závorkami a speciálními znaky, např. [- ^]
 - **Předdefinované skupiny znaků**, např. \w \W \s \S \d \D \
 - **Kotvy**: první nebo poslední shoda pro každý řádek, tj. ^ a \$
 - **Kvantifikátory**: počty opakování ve složených závorkách {.....} včetně rozsahů, uzávěry, nenulového a párového výčtu ? (0 nebo 1 shoda)
 - **Metaznaky** (Escape): ignoruje význam následujícího speciálního znaku
-
- Prostředí pro experimentování s RE: <https://regex101.com/>
 - RE cheatsheet: <https://www.rexegg.com/regex-quickstart.php>

Příklady regulárních výrazů

Expression	Matches
abc	abc
abc*	ab abc abcc abccc ...
abc+	abc, abcc, abccc, abcccc, ...
a(bc)+	abc, abcbc, abcbcbc, ...
a(bc)?	a, abc
[abc]	one of: a, b, c
[a-z]	any letter, a through z
[a\-z]	one of: a, -, z
[-az]	one of: - a z
[A-Za-z0-9]+	one or more alphanumeric characters
[\t\n]+	whitespace
[^ab]	anything except: a, b
[a^b]	a, ^, b
[a b]	a, , b
a b	a, b

Zdroj: <http://epaperpress.com/lexandyacc/prl.html>

LEX = Nástroj pro generování lexikálních analyzátorů

1. Lexikální analýza se realizuje **konečným automatem**.
2. **Gramatiky**, generující lexikální elementy se nazývají **regulární** nebo **lineární**.
3. **Regulární výrazy** reprezentují další možnost zápisu generující struktury.

Taxonomie

LEXEM: lexikální element (**konkrétní** vstupní prvek, vyhovující specifikaci jazyka)

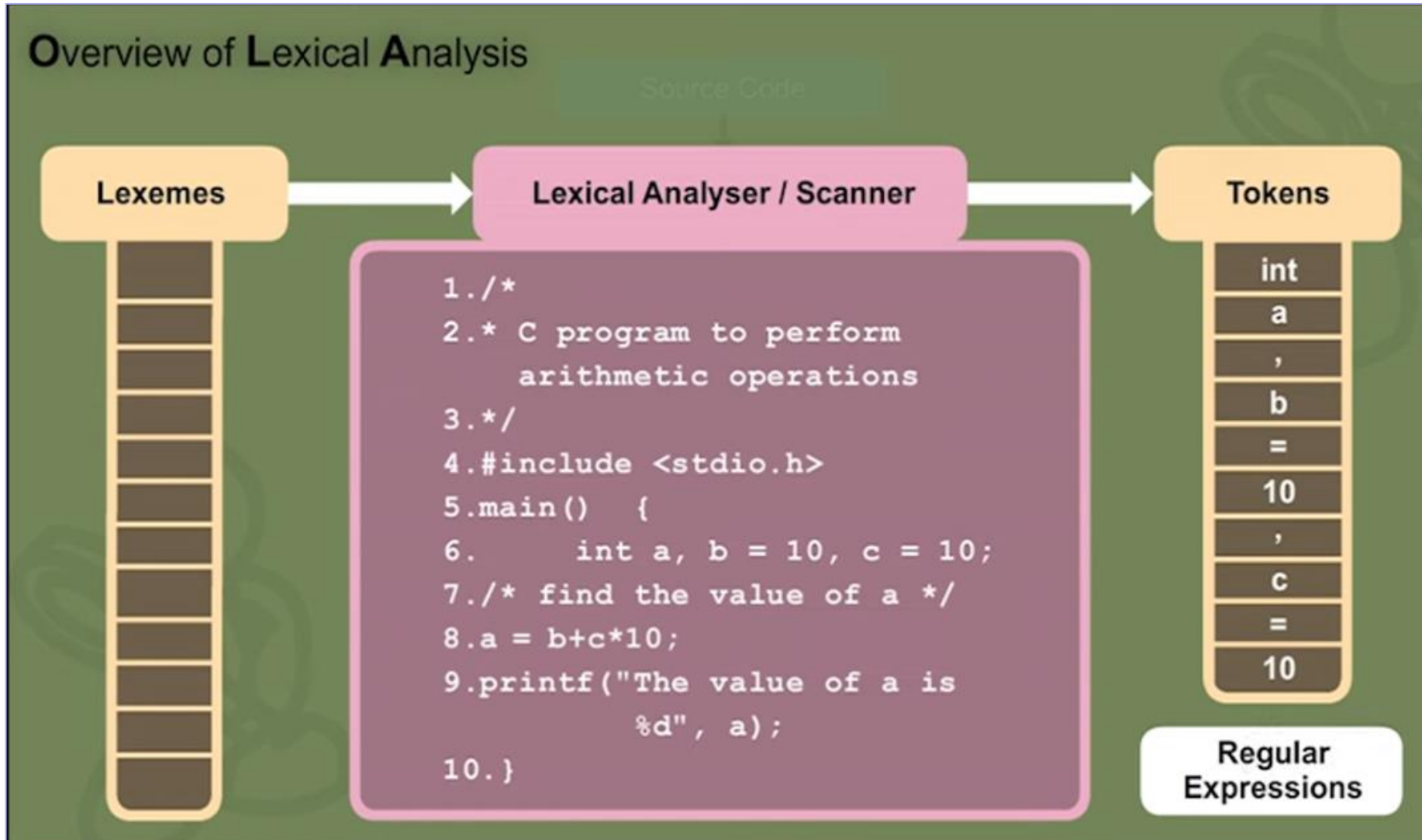
- Lexémy píše/vymýšlí/generuje programátor

PATTERN: **abstraktní** vzor, typicky regulární výraz, pokrývající více lexémů

- Vzory jsou pevnou součástí jazyka. Programátor nemůže při tvorbě zdrojového kódu použít konstrukci, kterou mu syntaxe (šablona) jazyka nedovolí.

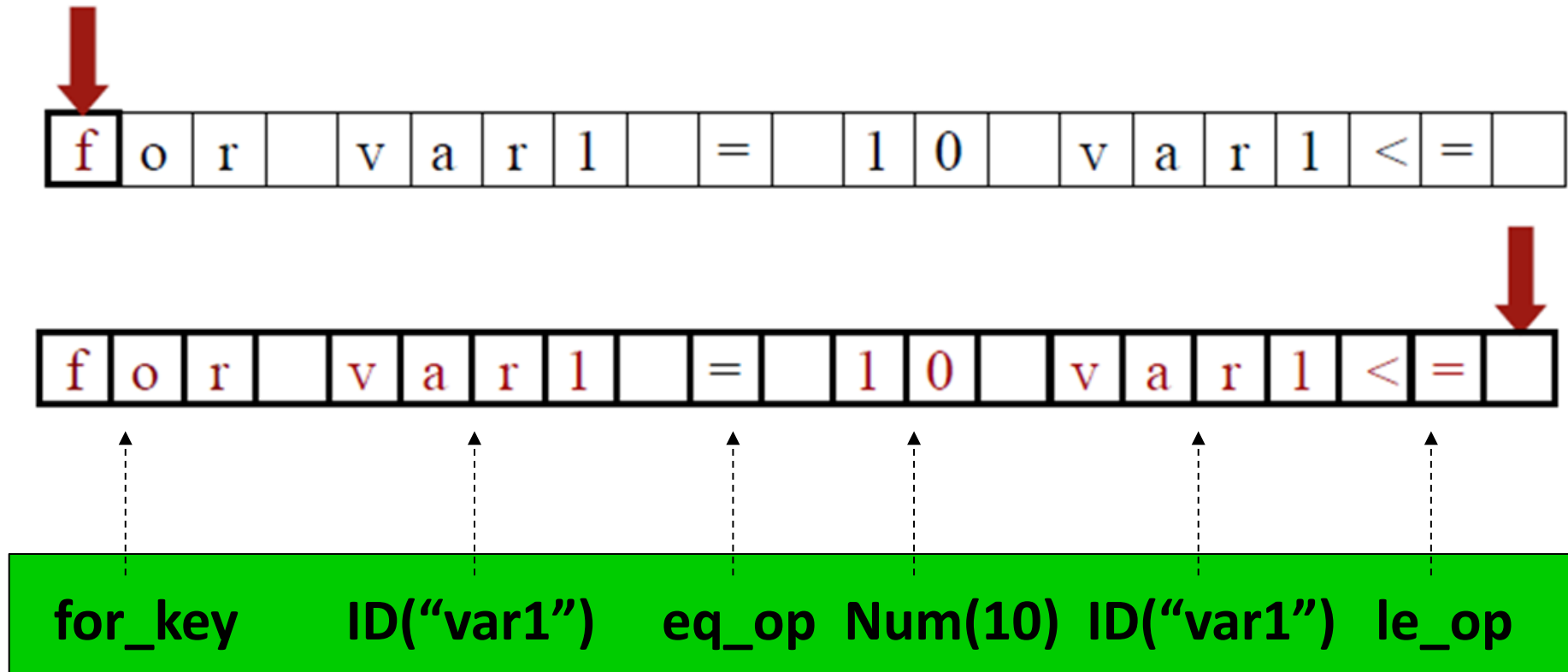
TOKEN: **gramatická** entita, vzniklá poté, co je lexem identifikován vzorem

- Token je jedním z výstupů lexikální analýzy (LA). Více lexémům může odpovídat jediný token (např. NUM, ID). Druhým typickým výstupem LA je konkrétní hodnota, odpovídající tokenu (např. 24, 8.5, i, count). Ta se ukládá do tzv. tabulky symbolů.



Ukázka lexikální analýzy 1

Vstup: sekvenčně skenovaná sada znaků



Výstup: sada tokenů

Ukázka lexikální analýzy 2

Vstup

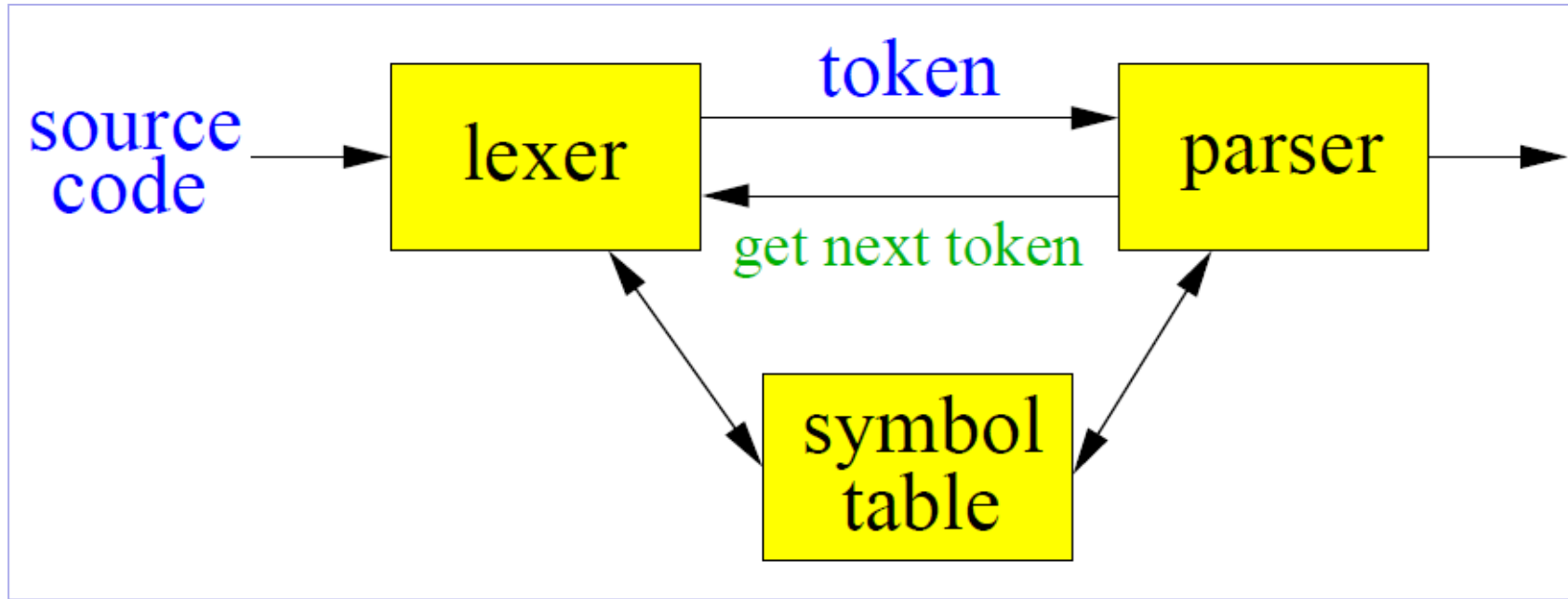
```
var = 12 + 9;  
if (test > 20)  
    temp = 0;  
else  
    while (a < 20)  
        temp++;
```

→ **Lex** →
./prog

Výstup

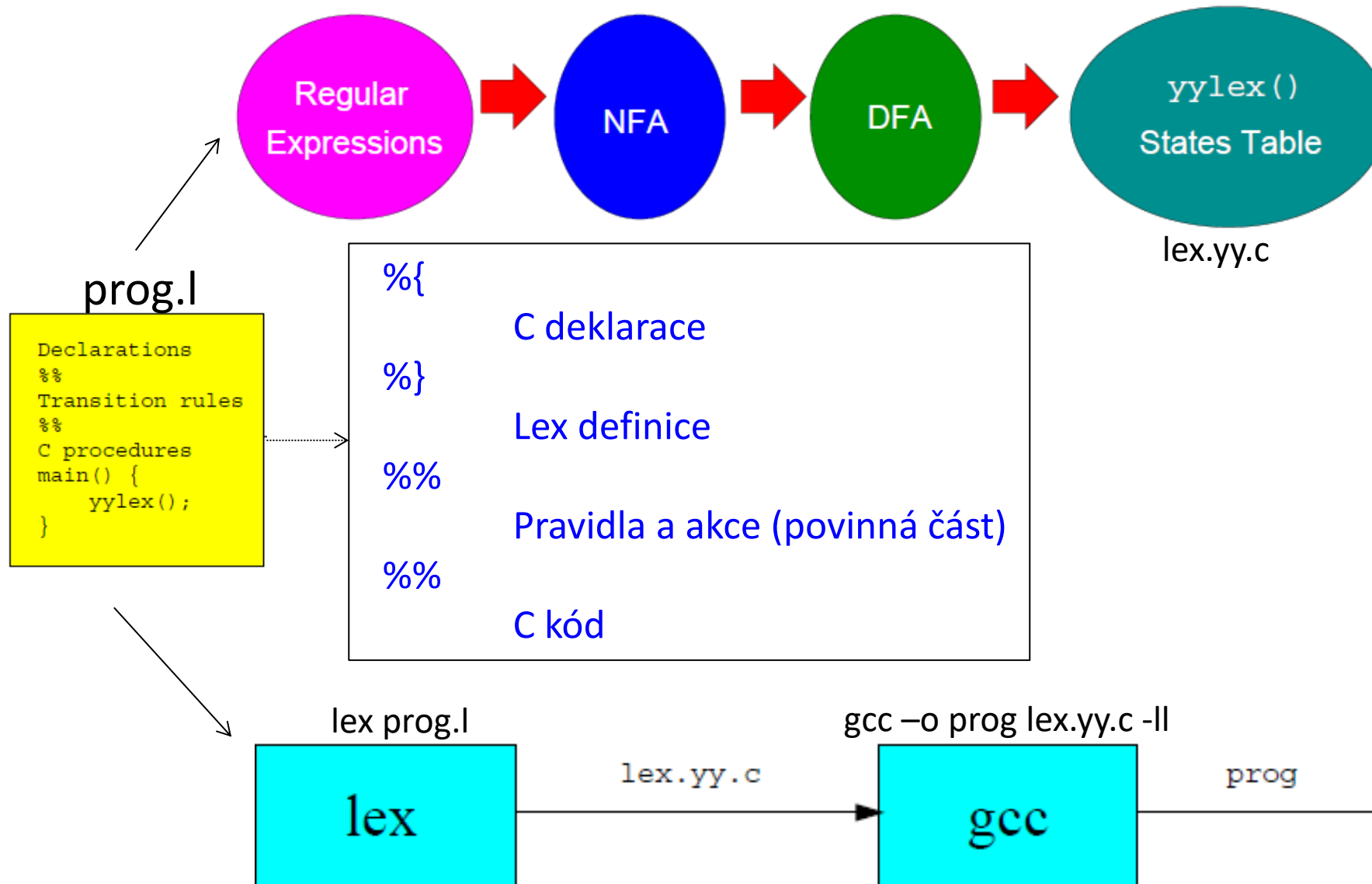
Identifier:	var
Operator:	=
Integer:	12
Operator:	+
Integer:	9
Semicolon:	;
Keyword:	if
Parenthesis:	(
Identifier:	test
....	

lex v kontextu dalších překladačových struktur



- „Tokenizovatelné“ lexemy se tokenizují, ty ostatní se z hlediska syntaxe ignorují (komentáře, řídicí znaky, textové literály, všechny typy mezer, ...)
- Konkrétní hodnoty tokenů a literálů se jako jeden z možných atributů ukládají do tabulky symbolů

lex, generátor lexikálních analyzátorů



Úkoly na příští týden

- Seznámit se s přednáškovými poznámkami a souvisejícími materiály
- Rezervace tématu individuální přednáškové prezentace do 5.3.
- Prezentace:
 - Hlídat si průběžně datum svého vystoupení v Excelové tabulce (data bývají aktualizována nejpozději do pátku)
 - Kdo prezentuje příští týden, nezapomene včas odevzdat svůj soubor